

MCP Core Concepts

Part 09 — 🦋 Model Context Protocol · 09-mcp/notes/01-core-concepts.md

Prereq: Module 04 — LLMs [04-llm] (tool calling basics) and Module 06 — AI Agents [06-ai-agents] (why a model needs tools at all). This note assumes you already know what a "tool call" is; it's about the protocol that standardizes how a model gets tools, data, and prompt templates in the first place.

1. The problem MCP exists to solve: $M \times N$ integrations

Before MCP (Anthropic open-sourced it in November 2024), every AI application that wanted to connect to external systems wrote its own integration. Claude Desktop's Google Drive connector, ChatGPT's Slack plugin, your internal agent's Postgres tool — each was bespoke code, with its own auth handling, its own way of describing what it could do, its own error conventions. If you have M AI applications and N systems they might want to reach (databases, SaaS APIs, filesystems, internal tools), you need up to $M \times N$ distinct integrations, each maintained separately, each breaking independently when either side changes.

This is the exact shape of problem that standard protocols solve elsewhere:

- Before USB, every peripheral needed its own port and driver. USB gave device makers one interface to build against and computer makers one port to support — an $M+N$ problem instead of $M \times N$.
- Before the Language Server Protocol (LSP), every editor had to write its own integration for every language's autocomplete, go-to-definition, and diagnostics. LSP let one language server work with any editor that speaks the protocol.

MCP's own docs use exactly the USB analogy: "Think of MCP like a USB-C port for AI applications." The promise is the same trade: a tool/data provider builds **one** MCP server; every MCP-speaking application (Claude, ChatGPT, an IDE, your own agent) can use it without bespoke glue code. You still write the adapter between MCP and your backend once — MCP just standardizes the interface so that adapter is reusable.

Where this can mislead you: MCP does not eliminate integration work — it relocates it. Someone still has to write the server that maps "search_catalog" to your actual database query, and someone still has to decide what a model should be allowed to do through that server. MCP standardizes the *wire contract*, not the judgment calls about scope, auth, and safety. Module 09's lab and §2 of notes/02 are about exactly those judgment calls.

2. The three primitives, and the one distinction that matters more than their names

MCP servers expose functionality through three primitive types. Memorizing their names is easy; the useful thing to internalize is that **each has a different control locus** — a different answer to "who decides when this runs?"

Primitive	Who decides to invoke it	Rough web analogy	Side effects
Tool	The model , autonomously, mid-conversation	POST endpoint	Expected — that's the point
Resource	The host application (or the user, by picking it)	GET endpoint	None — read-only by convention
Prompt	The user , explicitly, like a slash command	A saved template / menu item	None — it renders text

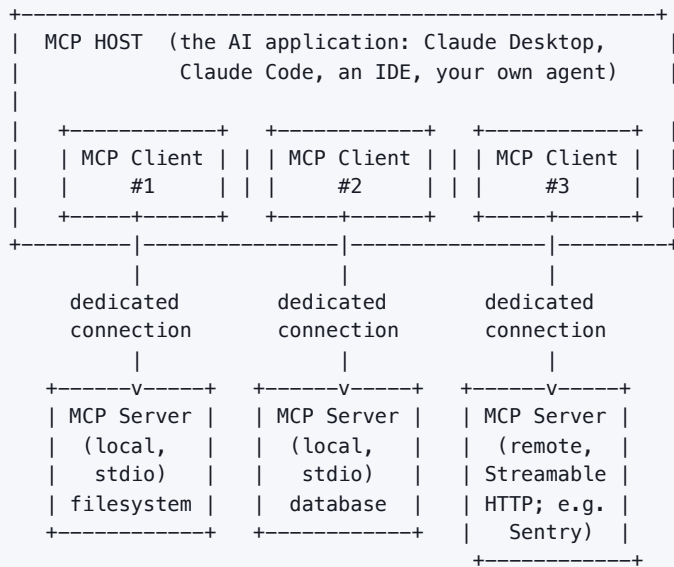
- **Tools** are executable functions the model can call to take an action or fetch something computed: search a catalog, run a query, send an email. The model reads the tool's `name`, its natural-language `description`, and its JSON Schema `inputSchema` to decide *whether* to call it and *how* to fill the arguments. This means a tool's description is not documentation for a human reader — it is the interface the model reasons over. A vague description ("process the request") makes the model guess; a specific one ("search the book catalog by title, author, or genre; case-insensitive substring match") tells it exactly when this tool applies and what a `query` value should look like.
- **Resources** are addressable, read-only context — file contents, a database schema, an API response — identified by a URI (e.g. `books://Dune`, `file:///project/README.md`). The model never "calls" a resource; the *host* decides when to read one and place its contents in front of the model, because a user clicked it, because it's pinned in a project config, or because the host's own retrieval logic selected it. `resources/list` is cheap and does not execute anything — only `resources/read` for a specific URI runs server code.
- **Prompts** are reusable, parameterized templates for starting an interaction — a prewritten "explain this code" or "recommend a book in this genre" that appears as a slash command or menu item in the client UI. A person picks it, fills in the parameters, and the rendered text is inserted into the conversation as if they had typed it.

The most common MCP design mistake is mixing these up: putting a pure data-fetch behind a Tool (now the model has to *decide* to fetch context that should have just been loaded for it, wasting a turn and risking it never asking) or putting a state-changing action behind a Resource (nothing stops a host from silently loading a "resource" into every conversation — resources are specified to be read without side effects, so if yours has one, you've violated the contract and something downstream will eventually trust that assumption and get burned).

There is a second, smaller set of primitives that flow the *other* direction — a server asking the client for something. **Elicitation** lets a server request additional structured input from the user mid-task (e.g., "which of these three accounts did you mean?"). Two older client primitives, **sampling** (server asks the client's LLM to generate a completion) and **logging** (server sends log messages to the client), were marked **deprecated** in the 2026-07-28 spec revision — new servers should call an LLM provider directly instead of routing through sampling, and should log to `stderr` (`stdio`) or `OpenTelemetry` instead.

3. Architecture: Host, Client, Server

MCP is a client-server protocol with a specific three-role vocabulary that the spec is precise about:



- **MCP Host** — the AI application the human actually uses. It coordinates one or more clients, decides which servers to connect to, and ultimately puts tool results and resource contents in front of the model.
- **MCP Client** — one per server connection. The host instantiates a fresh client for each server it talks to; a client maintains that one dedicated connection and does nothing else. In `code/mcp_client_demo.py`, `ClientSession` is the client.
- **MCP Server** — the program that actually implements tools/resources/prompts against some backend (filesystem, database, SaaS API). "Server" describes the role, not where it runs: a **local** server (stdio transport, spawned as a subprocess, typically serving exactly one client) and a **remote** server (Streamable HTTP, typically serving many clients at once) are both "MCP servers" in spec terms.

This 1:1 client-to-server pairing is deliberate — it's why a host that's connected to five MCP servers has five client objects, each blind to the others' state, which is also why cross-server coordination (e.g., "use the result from server A as an argument to server B") is the *host's* job, not something the protocol does for you.

4. The wire protocol: JSON-RPC 2.0, and two eras of how a session starts

Every MCP message, regardless of transport, is a [JSON-RPC 2.0](#) object: a request has `jsonrpc`, `id`, `method`, `params`; a response has `jsonrpc`, `id`, and either `result` or `error`; a notification is a request with no `id` and therefore no expected reply. The SDK's pydantic models (`InitializeResult`, `CallToolResult`, etc.) are literally that JSON, deserialized — `model_dump(mode="json")` on any of them reproduces the wire payload byte-for-byte (see `mcp_client_demo.py`'s `show()` helper).

The classic era (spec revisions 2024-11-05 through 2025-11-25 — what you will meet almost everywhere today): a session opens with an explicit handshake:

1. Client -> Server: `initialize` request, carrying the client's supported `protocolVersion`, its `capabilities` (which optional features it supports — e.g. `roots`, `sampling`), and `clientInfo`.

2. Server -> Client: replies with its own `protocolVersion` (possibly older, if it doesn't support the client's), its `capabilities` (which primitives it exposes — `tools`, `resources`, `prompts` — and whether each supports `listChanged` notifications), `serverInfo`, and optional human-readable `instructions`.
3. Client -> Server: `notifications/initialized` — a one-way notification, no reply expected. The session is now in the "operation" phase and normal requests (`tools/list`, `tools/call`, `resources/read`, ...) can flow.

This is exactly what `mcp_client_demo.py` does and prints: `session.initialize()` performs steps 1-3 in one call, and the negotiated version you'll see in its output is `2025-11-25` (mcp SDK 2.2.0's newest supported classic revision) — the SDK negotiates downward if a server it's talking to only understands an older one.

Statelessness and discovery (2026-07-28 — the current spec as of this writing, adoption still nascent): the `initialize/initialized` handshake was removed entirely, along with the protocol-level session concept (the `Mcp-Session-Id` header) that Streamable HTTP used to carry. In its place, MCP is now specified as a genuinely stateless protocol: **every** request carries its own `protocolVersion` and `capabilities` in a `_meta` field, so a server can process each request in isolation without remembering a prior handshake. Discovery becomes a plain request-response call, `server/discover`, that a client *may* send before anything else (it's optional precisely because every other request is already self-describing):

```
// Request
{"jsonrpc": "2.0", "id": 1, "method": "server/discover",
 "params": {"_meta": {"io.modelcontextprotocol/protocolVersion": "2026-07-28",
                    "io.modelcontextprotocol/clientInfo": {"name": "example-client", "version":
"1.0.0"},
                    "io.modelcontextprotocol/clientCapabilities": {"elicitation": {}}}}}

// Response
{"jsonrpc": "2.0", "id": 1,
 "result": {"resultType": "complete", "supportedVersions": ["2026-07-28"],
           "capabilities": {"tools": {"listChanged": true}, "resources": {}},
           "_meta": {"io.modelcontextprotocol/serverInfo": {"name": "example-server", "version":
"1.0.0"},
                    "ttlMs": 3600000, "cacheScope": "public"}}
```

Two other consequences of this redesign worth knowing even if you're not building against it yet: list results (`tools/list`, etc.) are now explicitly cacheable via `ttlMs`/`cacheScope` fields on every response, and cross-call state (a shopping cart ID, a workflow handle) is no longer implicit session state — a server that needs it mints an explicit handle and receives it back as an ordinary tool argument on later calls, which the security notes in [notes/02](#) cover (state-handle hijacking is a named attack in the 2026-07-28 security guidance precisely because of this shift).

Practical takeaway: the `mcp` Python SDK (and the equivalent TypeScript SDK) is "dual-era" — it still fully speaks the classic handshake, because that's what virtually every deployed server, IDE integration, and tutorial uses today. Build and test against the classic protocol unless you have a specific reason to target a bare 2026-07-28 stateless server; the SDK insulates you from most of this either way.

5. Transports: stdio vs. Streamable HTTP

The data layer (JSON-RPC messages, primitives, capabilities) is transport-agnostic; the transport layer just carries those same messages over a different pipe.

Transport	How it works	Typical use	Notes
stdio	Client spawns the server as a subprocess ; messages are newline-delimited JSON-RPC on the child's stdin/stdout	Local servers: filesystem access, a local database, a CLI wrapper	No network overhead; the server inherits the client process's privileges — see the security note in notes/02
Streamable HTTP	Client <code>POST</code> s JSON-RPC to an HTTP endpoint; the server can respond with a single JSON reply or open a Server-Sent Events stream for multiple/async messages	Remote, shared servers: a SaaS's official MCP server (e.g. Sentry), an internal server many users hit	Supports standard HTTP auth (bearer tokens, API keys, OAuth)

An older transport, HTTP+SSE (two separate endpoints), was replaced by Streamable HTTP in the 2025-03-26 revision — if you see it in an older tutorial, treat it as deprecated. `code/minimal_mcp_server.py` uses `stdio` (`mcp.run(transport="stdio")`) because it's meant to be spawned by `code/mcp_client_demo.py` as a child process — the default and simplest way to run a server you own and only you (or your local host app) will use.

6. Building a server: naming, descriptions, schemas, errors

The mechanics, worked through concretely in `code/minimal_mcp_server.py`:

- **Tool names** should be specific verbs, not generic ones — `search_catalog`, not `handle` or `process`. The 2025-11-25 revision added explicit spec guidance on tool naming for exactly this reason: names and descriptions are the model's *only* interface into what a tool does, so ambiguity there directly causes tool mis-selection.
- **Descriptions are load-bearing**. Write them for the model, not for a human skimming your code. State the matching semantics ("case-insensitive substring match"), not just the verb.
- **Schemas come from your code, not a separate file to keep in sync**. The Python SDK's decorator-based API builds the JSON Schema from your function's type hints automatically — this is the actual value of using an SDK instead of hand-rolling JSON-RPC: schema drift between "what the model is told" and "what the function accepts" becomes structurally impossible.
- **Errors are a first-class part of the interface, not an afterthought**. A tool that hits a bad input should raise a `ToolError` (or your SDK's equivalent), which the protocol turns into a **result** with `is_error=True` and a message in `content` — the call still *succeeds* at the JSON-RPC level. This is deliberate: the model reads that message and can retry with better arguments. If you instead `return` an error string, every client reads `is_error=False` and treats it as a successful answer — the single most common bug in hand-rolled MCP tools. Resources behave differently: a missing resource is a genuine protocol-level error

(`ResourceNotFoundError` -> JSON-RPC error code `-32602`), because there's no "succeeded but flagged" path for a read that found nothing.

7. Where this returns

Topic	Comes back in
Tool-calling mechanics, why the model "decides" to call a tool	Module 06 — AI Agents [06-ai-agents]
Tool namespacing and catalogs at scale (many servers, name collisions)	notes/02, §2 of this module
MCP vs. A2A (agent-to-agent protocols)	Module 07 — Agentic AI [07-agentic-ai/notes/02-coordination-control-and-evaluation.md] and notes/02 §5 here
MCP server as a trust boundary / third-party dependency	Module 13 — AI Security [13-ai-security] and notes/02 §3-4 here
Vendor/MCP risk in a governance program	Module 12 — AI Governance [12-ai-governance]
MCP as the tool layer in a reference architecture	Module 10 — AI Architecture [10-ai-architecture]
Practical day-to-day MCP config in a real agent	Module 16 — Claude Code track [16-ai-tech-stack/tracks/claude-code]